GenAI & große LLMs



Heute hier vor Ort



Jennifer Lucifero



Wer seid ihr?

Wer hat schon einmal von „Large Language Models“ oder „LLMs“ gehört?

Wer hat schon einmal selbst ein KI-Tool wie ChatGPT, Gemini oder Copilot ausprobiert?

Wer weiß, wie ein LLM grundsätzlich funktioniert?

Wer arbeitet regelmäßig mit KI-Tools oder LLMs im Beruf?

Wer hat schon einmal ein LLM auf dem eigenen Computer oder Server laufen lassen?



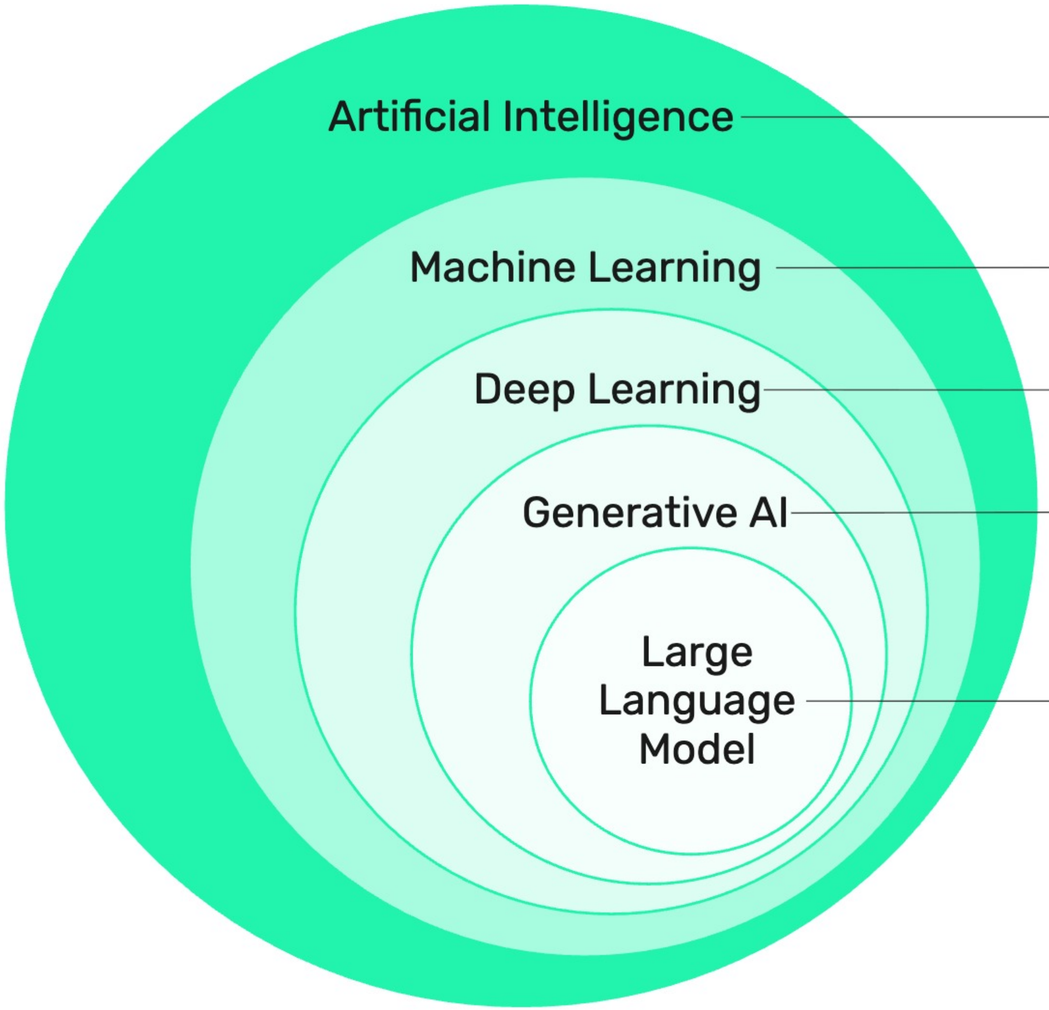
Die Illusion der Intelligenz

Eine LLM weiß nichts - es schätzt nur verdammt gut.

Kurzes Inference-Spiel

"Wer im Glashaus sitzt ..."

Begrifflichkeiten



AI: Intelligente Maschinen (seit 1950s)

ML: Pattern Recognition - Lernt generelle Patterns aus Daten (seit 1980s)
DL: Neuronal Networks - Lernt generelle Pattern aus unstrukturierten Daten z. B. Bilder, Text, Audio, etc. (seit 2000s)

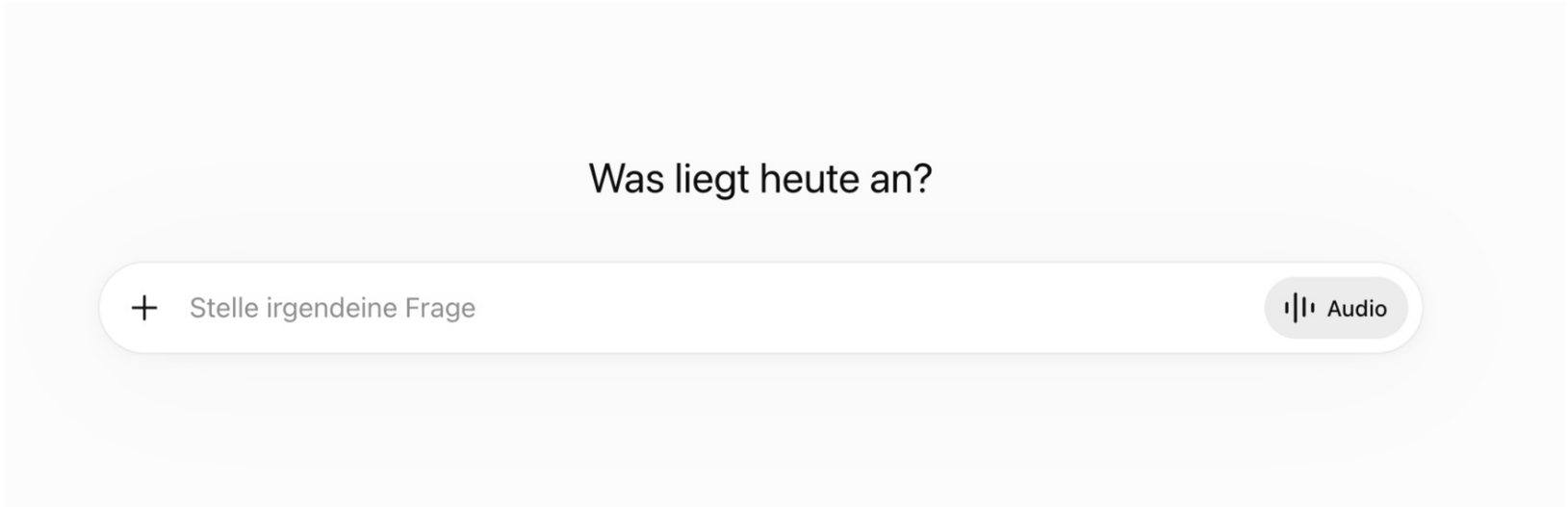
GenAI: kann neuen Content generieren (seit 2022)

LLM: lernt natürliche Sprache zu verstehen (seit 2020s)



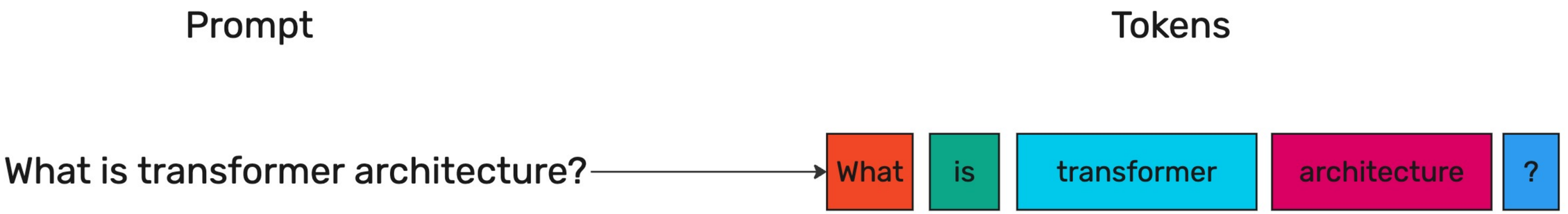
Wie funktioniert ein LLM überhaupt?

Das kennen wir alle ...



Transformer Architektur

1. Tokenization

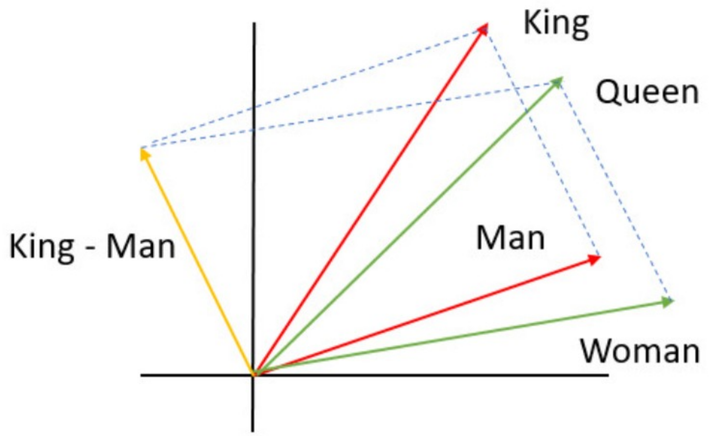


Transformer Architektur
2. Embedding

👀

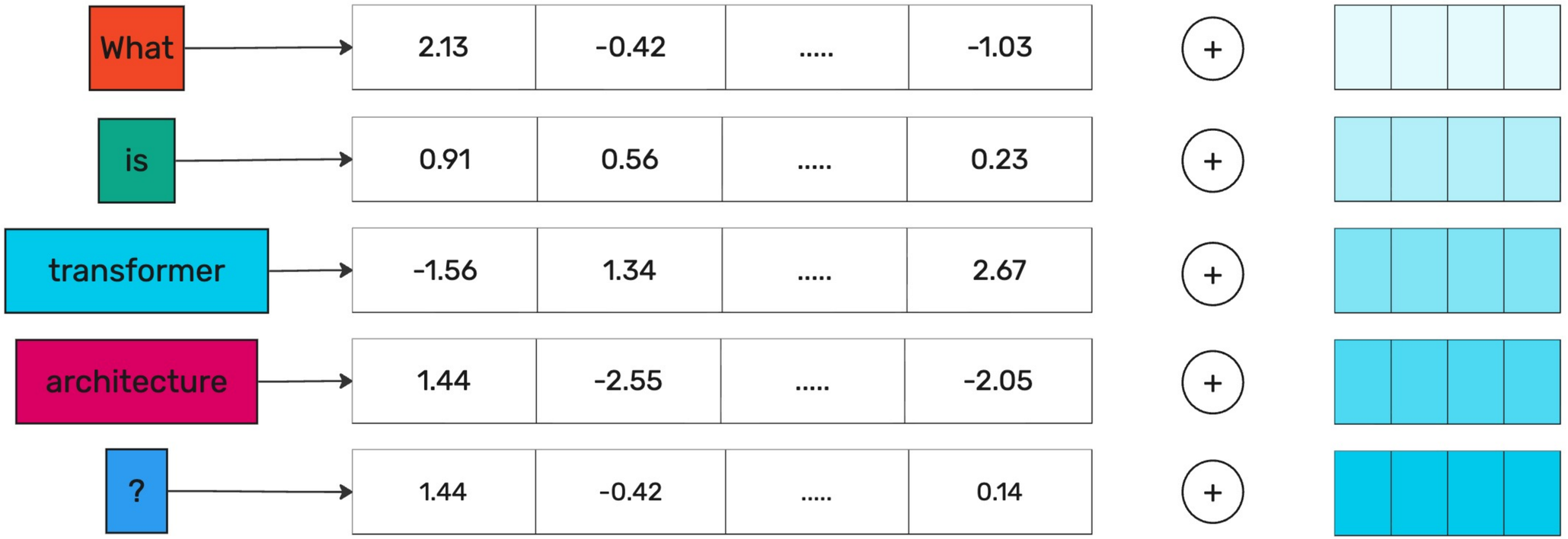
"man bites dog"
==
"dog bites man"

What	2.13	-0.42	.....	-1.03
is	0.91	0.56	.....	0.23
transformer	-1.56	1.34	.....	2.67
architecture	1.44	-2.55	.....	-2.05
?	1.44	-0.42	.....	0.14



Transformer Architektur
3. Positional Encoding

"man bites dog"
!=
"dog bites man"

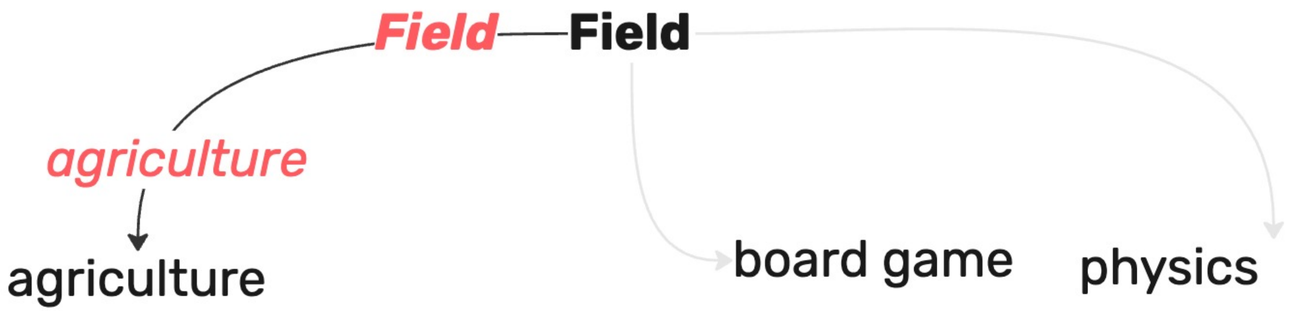


Transformer Architecture

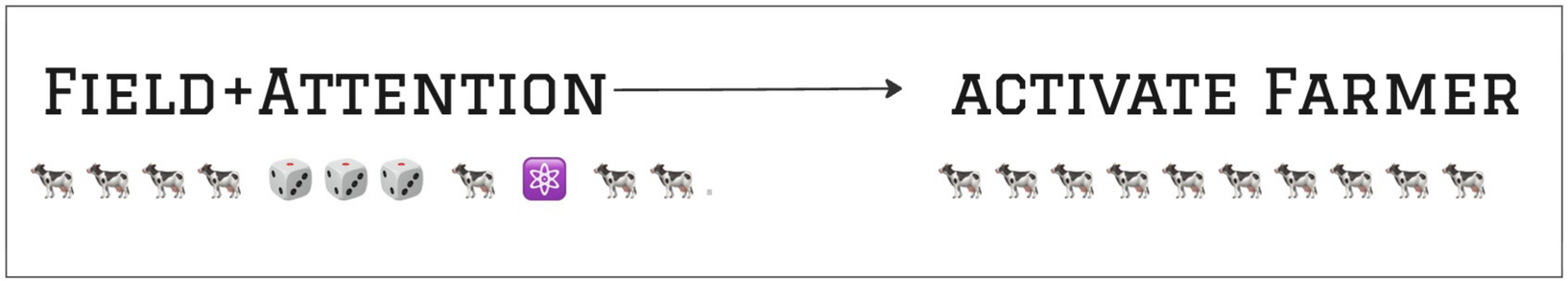
4. Self-Attention & Feedforward neural network

The farmer tills his **field** for the next harvest

"Attention"

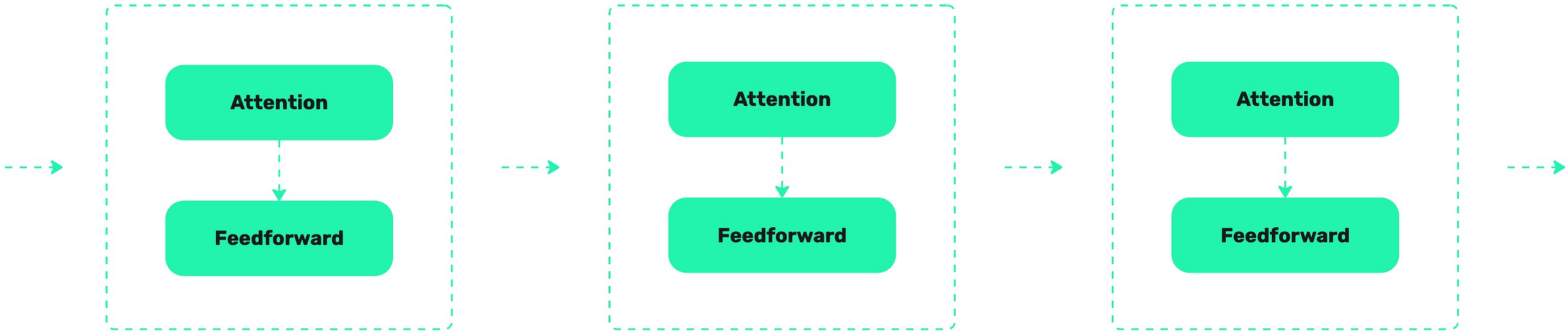


"Feed-Forward"



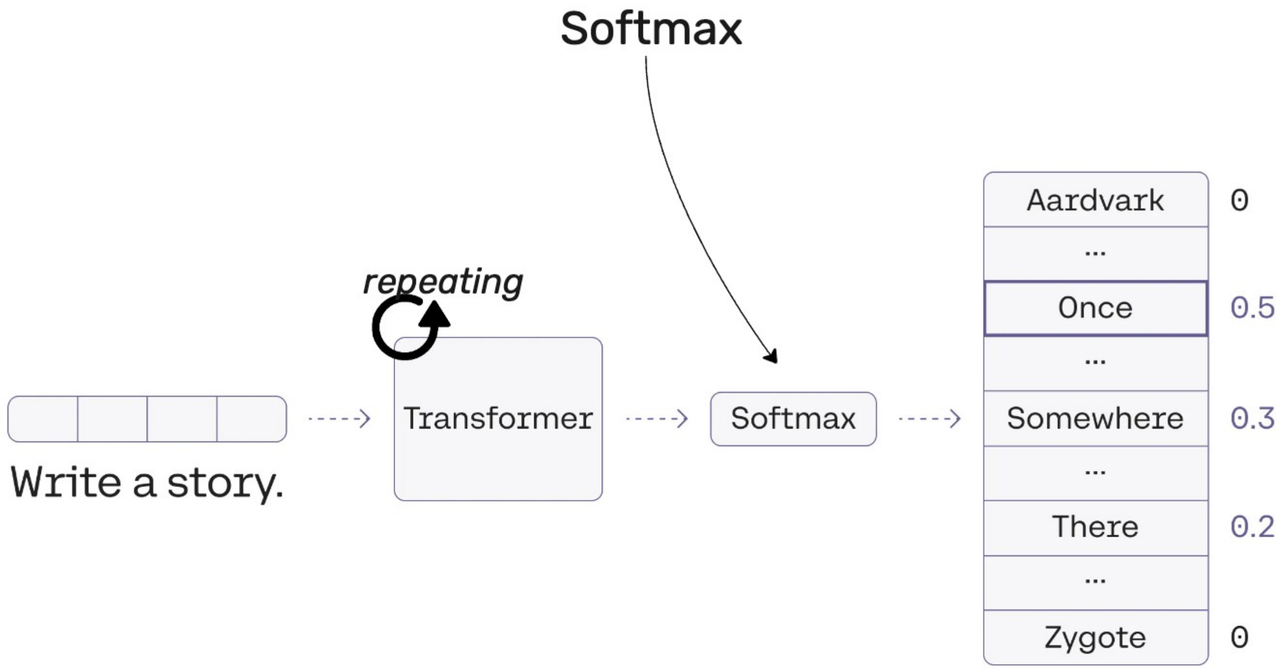
Transformer Architektur

4. Self-Attention & Feedforward neuronal network



Transformer Architektur

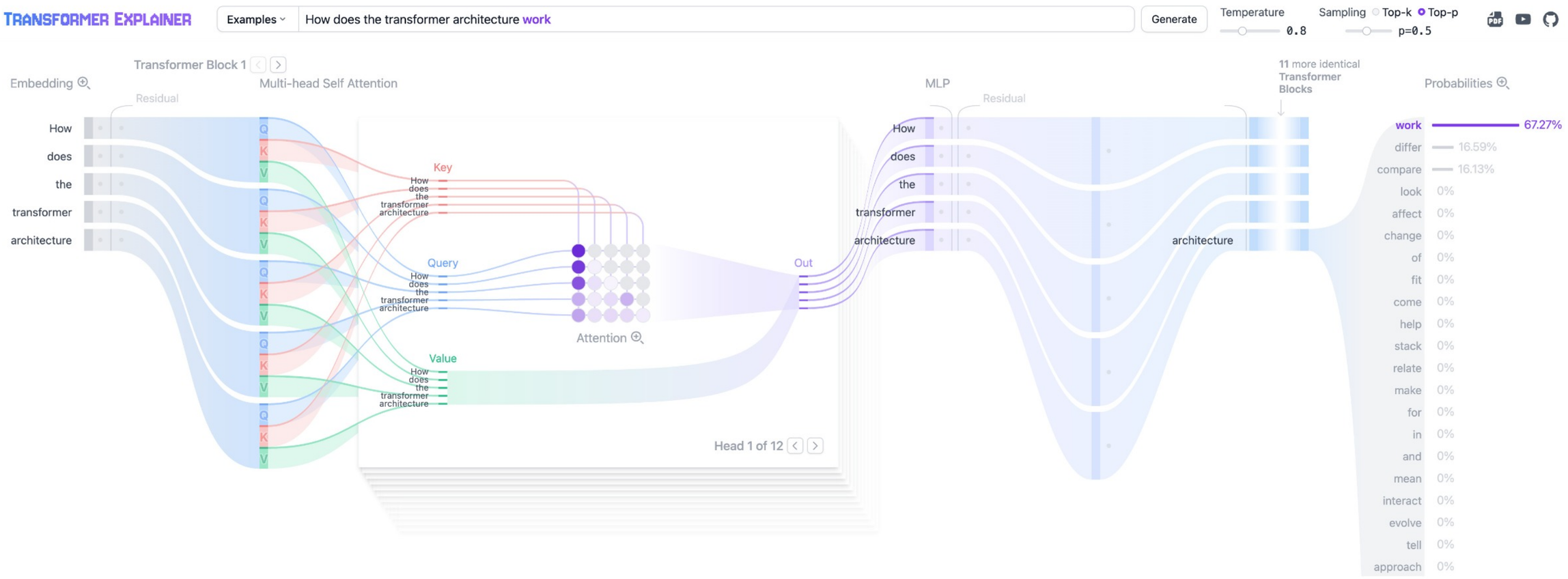
5. Softmax – for the human touch



The system does not always select the token with the highest ranking, but randomly picks one from the highest ones.

Tests have shown that this appears more “human.”

Transformer Architektur
Visualisierung



Source: https://poloclub.github.io/transformer-explainer/



Warum groß?
Was macht eine LLM groß?

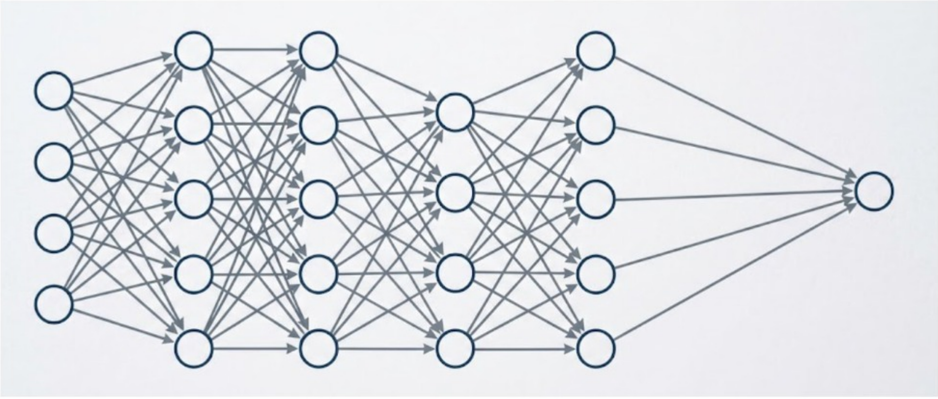
Was macht eine LLM groß?

- Kurzantwort: Anzahl der Parameter
- 405 Milliarden trainierbare Parameter > 7 Milliarden trainierbare Parameter

Para … was? Die Drehregler-Metapher

Parameter = Milliarden kleiner Regler

Diese Regler bestimmen, wie das Modell funktioniert und welche Ausgabe erzeugt wird.





Gesamtparameter vs Aktive Parameter

- Mixture of Experts (MoE)
  - Ein Mixture of Experts (MoE) LLM ist ein Modell, das aus vielen spezialisierten "Experten"-Netzwerken besteht, wobei für jede Eingabe nur die relevantesten Experten aktiviert werden
  - Hohe Kapazität, aber günstiger

- **Mixtral 8x7B**
  - 8 Experten mit je ~7 Milliarden Parametern

Führende Modelle im Vergleich

Anbieter	Bekanntestes Modell	Bekannte / Geschätzte Größe
OpenAI	**GPT-4o**	**~1,8 Billionen** *(Geschätzt für GPT-4-Familie; neuere Varianten kompakter)*
Anthropic	**Claude 3.5 Sonnet**	**Keine offiziellen Angaben** *(Schätzungen: ~200B–400B)*
Google	**Gemini 1.5 Pro**	**Keine offiziellen Angaben** *(Schätzungen: ~1 Trillion+)*
Meta	**Llama 3.1 (405B)**	**405 Milliarden** *(Exakt)*
DeepSeek	**DeepSeek-V3 / R1**	**671 Milliarden** *(Exakt; 37B aktiv)*

Weitere Faktoren

- Menge und Qualität der Trainingsdaten
- Architektur
- Post-Training
- Inferenz bzw. Generierungsgeschwindigkeit
- Größe des Kontextfensters



Context Engineering

Prompts vs. Context

Prompt: Einzelne Anweisung für das LLM oder den Agent

Context: Alle Prompts, Tool Aufrufe, Gelesene Dateien, Compiler Ausgaben, Antworten des Modells, ...

Kontextgrößen moderner Modelle

Moderne Modelle haben eine enorme Kontextgröße

Claude 4.8 Opus: ~1M
GPT 5.4: ~1M
Gemini 3 Pro: 1M

Using o200k_base in tiktokenizer:

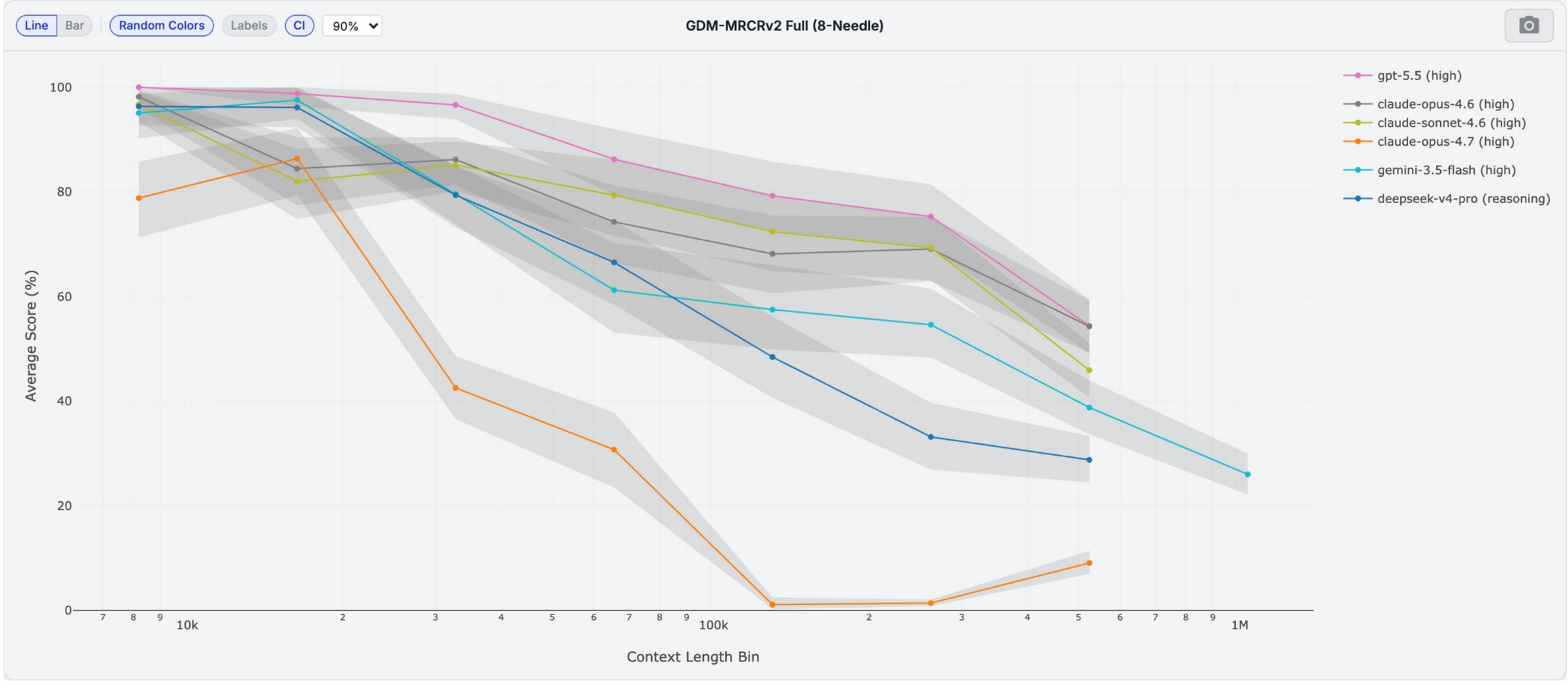
War and Peace: ~770k
Moby Dick: ~310k
The King in Yellow: ~103k
Alice in Wonderland: ~41k

Kontextgröße

Aber: **mehr** ist nicht direkt **besser**!

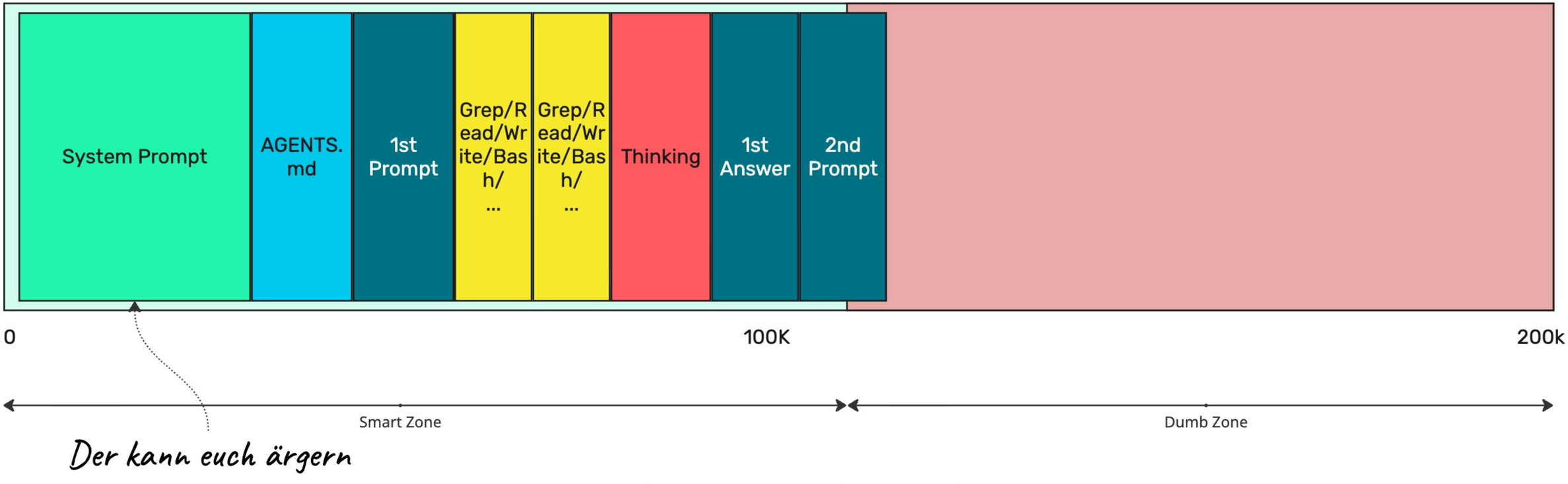


Performance sinkt mit Kontextgröße



https://contextarena.ai/

Beispiel für einen Kontext



Ziel: Arbeiten in der Smart Zone

Strategien für Context Management

- Immer geltende Anweisungen oder Regeln in Dateien auslagern
- Neustart bei Themenwechsel
- Antwortlänge begrenzen, expliziter Hinweis darauf "antworte kurz und prägnant"
- Kompression des bisherigen Stands
- Falls es sich nicht vermeiden lässt: Modelle mit großer Kontextgröße wählen



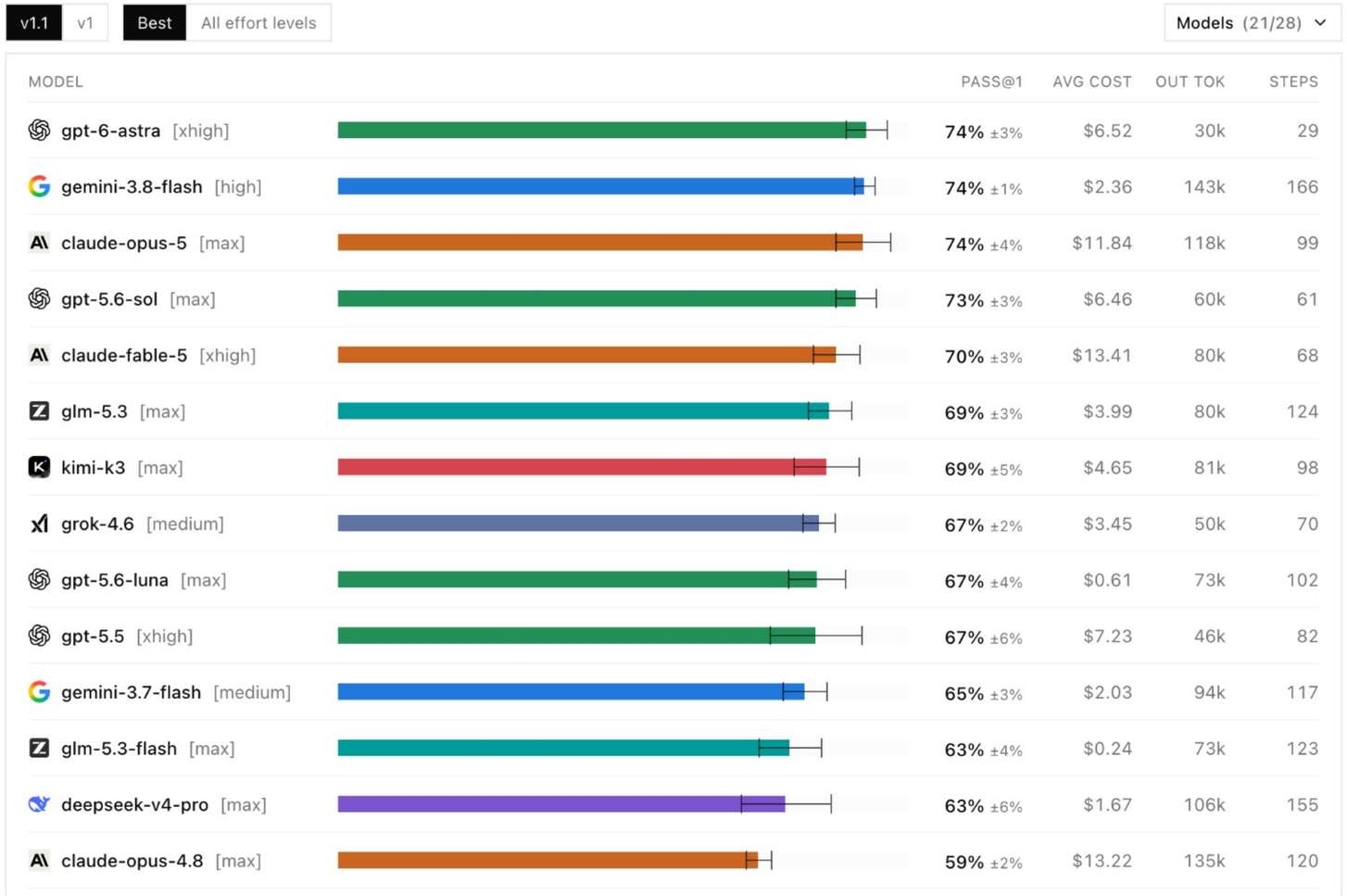
Modelle & Tools

Es gibt mittlerweile unzählige Modelle

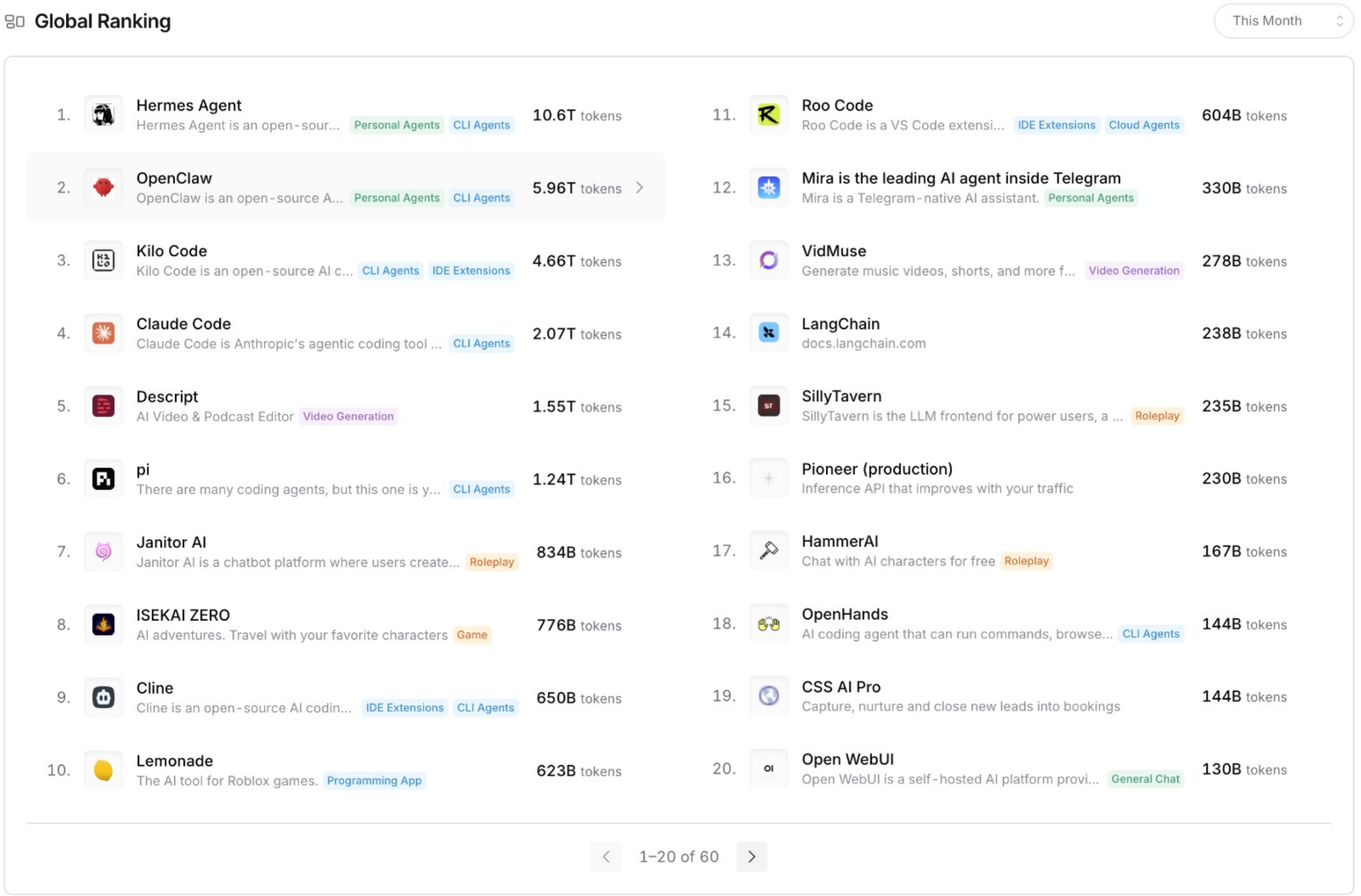
DeepSWE

DeepSWE

DeepSWE measures frontier coding agents on original, long-horizon software engineering tasks.



Und unzählige Tools



Also welches Modell nehm' ich?

Das ist **Geschmacksache**, aber es gibt Kriterien zur Orientierung

Preis pro Input/Output Token

Größe des Kontext-Fensters

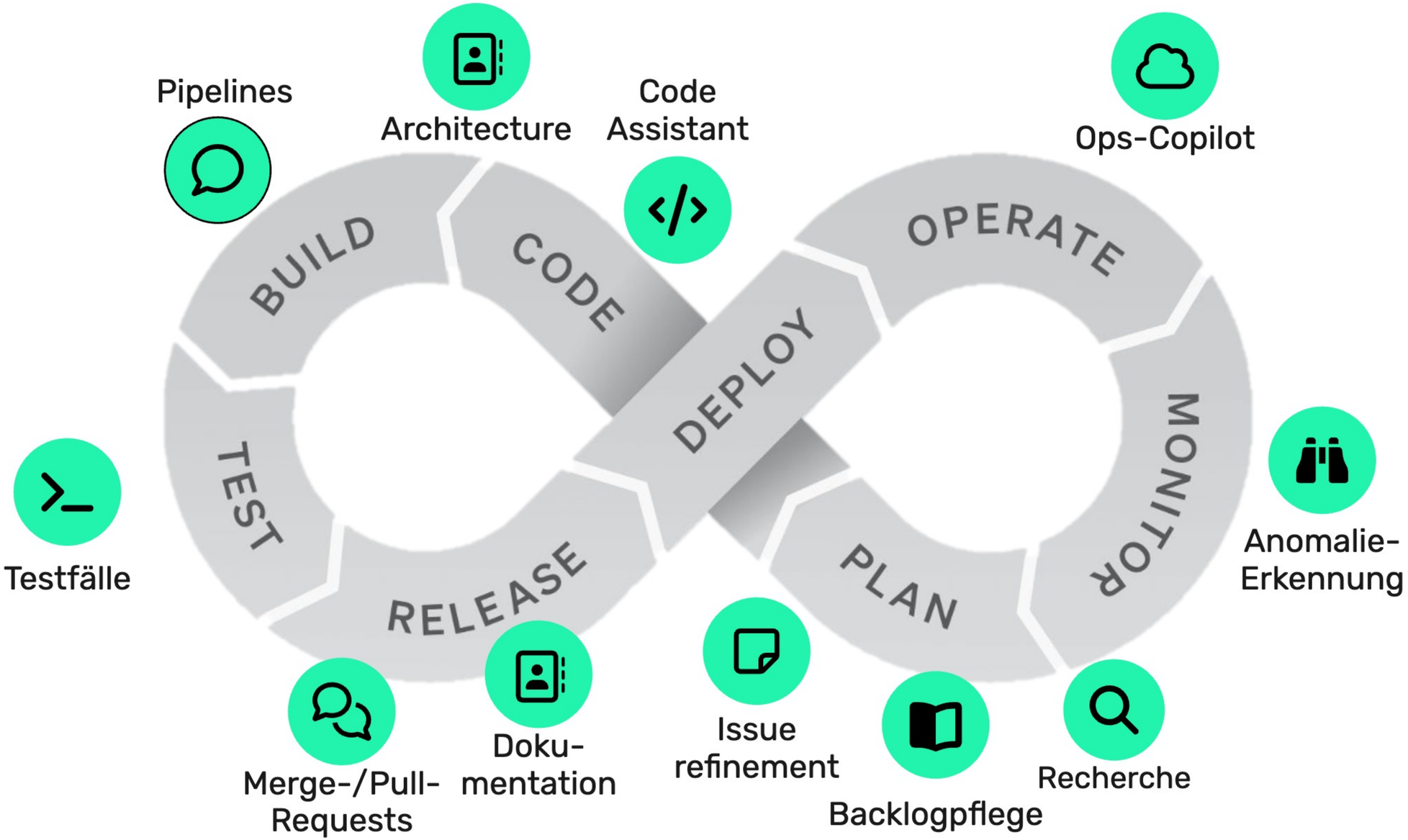
Inferenz-Geschwindigkeit

Außerdem kann es interessant sein, bewusst Modelle auszuprobieren, die entweder **quelloffen** sind, **lokal laufen** oder **aus der EU stammen**.



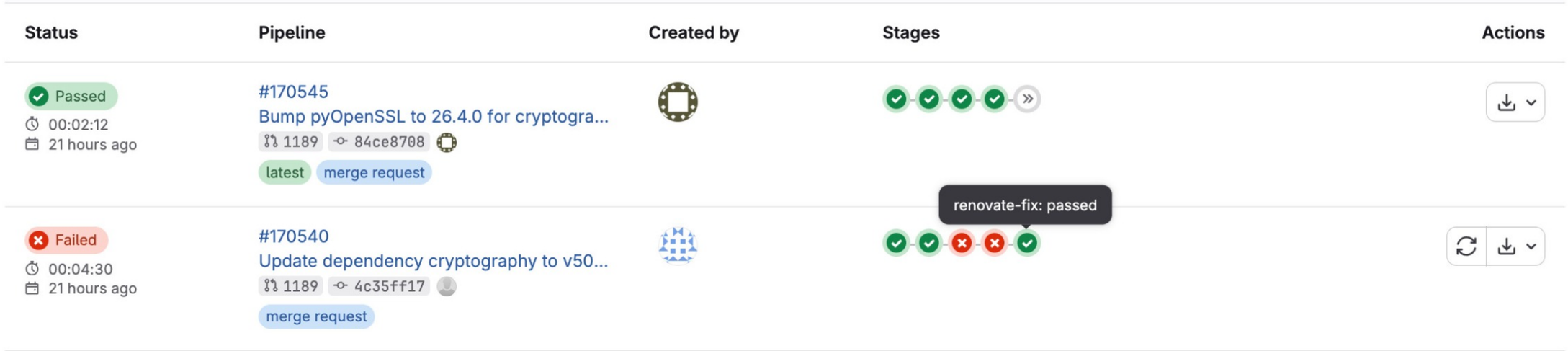
Anwendungs-beispiele im Alltag

DevOps-Schleife

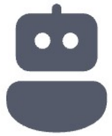


Beispiel: Renovate Fixer

- Reagiert automatisch auf fehlgeschlagene Renovate Pipeline

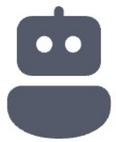


Beispiel: Renovate Fixer



Main Agent

- liest MR-Details, Commits, Pipeline-Logs
- Dateien im Branch lesen/schreiben
- Shell Befehle ausführen
- Lock-Files per Package Manager neu erstellen
- darf Fix-Commits pushen

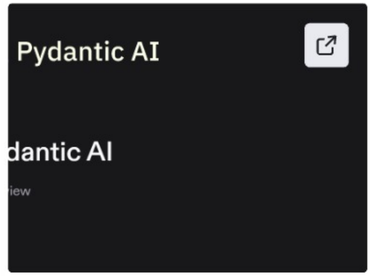


Research Agent

- wird augerufen vom Main Agent um Informationen zu Updates zu sammeln (Einstiegspunkt: Google Search, hat sonst Links halluziniert)
- Großer Kontext, weniger "schlau"
- Gibt Summary an Main Agent zurück

Beispiel: Renovate Fixer

```python
model = create_requesty_model(renovate_helper_settings.model)
gitlab_toolset = FastMCPToolset[DepedencyUpdateDeps](gitlab_mcp)
local_toolset = FastMCPToolset[DepedencyUpdateDeps](local_mcp)
agent = Agent(
    model=model,
    system_prompt=renovate_helper_settings.system_prompt,
    deps_type=DepedencyUpdateDeps,
    toolsets=[
        gitlab_toolset,
        local_toolset,
    ],
)
agent.tool(retries=3)(research_major_updates)
```



Pydantic Docs

Pydantic AI

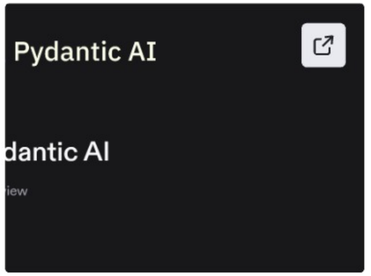
Pydantic AI is a Python agent framework designed to help you quickly, confidently, and painlessly build production grade applications...

Beispiel: Renovate Fixer

```
    system_prompt: str = f"""\
You are a Renovate merge-request fixer for GitLab.

Renovate bumped a dependency and the MR's pipeline broke. Figure out why and push
one fix commit to the MR's own branch. The repo is checked out on disk and you
have tools to inspect the MR, read and write files, and run shell commands; each
tool's own description tells you what it does.

The image is containerbase-based, so you can install any toolchain on demand with
`install-tool <name> <version>` (node, flutter, golang, rust, ...). The toolchain
preinstalled in the image is NOT the project's — never assume the `node`/`npm` (or
other tool) already on PATH is the right one. Before running any package-manager
command, find the exact versions the failing job uses (its CI image/config, and the
project's own pins) and `install-tool` those first. A lock file regenerated with a
different toolchain version usually passes locally but then fails the same way in
CI, wasting a whole pipeline retry.
```



Beispiel: SmartClaim AI

- SmartClaim AI ist eine interaktive Demo einer KI-gestützten Versicherungsschadensbearbeitung.

- Die App simuliert den vollständigen Bearbeitungsprozess eines Schadensfalls in fünf Schritten – von der Schadensmeldung bis zur automatisch generierten Kundenkommunikation.

- Jeder inhaltliche Schritt wird durch einen echten API-Aufruf an **Mistral AI** durchgeführt: https://gitlab.codecentric.de/codecentric/devk-demo

- **Wurde in 9h hauptsächlich durch Claude Sonnet gebaut**

Beispiel: SmartClaim AI

```python
response = requests.post(
    url: 'https://router.eu.requesty.ai/v1/chat/completions',
    headers={
        'Content-Type': 'application/json',
        'Authorization': f"Bearer {os.environ['MISTRAL_API_KEY']}"
    },
    json={
        'model': 'mistral/mistral-medium-latest',
        'messages': [{
            'role': 'user',
            'content': [
                {
                    'type': 'text',
                    'text': 'Du bist ein Schadensgutachter für eine Versicherung. Analysiere dieses Bild und beschreibe detailliert: 1) Wel
                },
                {
                    'type': 'image_url',
                    'image_url': {'url': image_data_url}
                }
            ]
        }],
        'max_tokens': 1024
    }
)
```



Fazit

Nachteile großer LLMs

Hoher Rechenaufwand und Energieverbrauch

Dadurch hohe Betriebskosten

Großer Speicherbedarf

Vorteile großer LLMs

Erfassen langer und komplexer Kontexte

Geeignet für einfache und auch komplexe Aufgaben

Umfangreiches Faktenwissen
Da sie auf großen Datensätzen trainiert wurden

Breite Anwendungsbereiche
Können sehr viele verschiedene Aufgaben lösen

Ausblick

- Der Wandel vom Chatbot zu Agenten

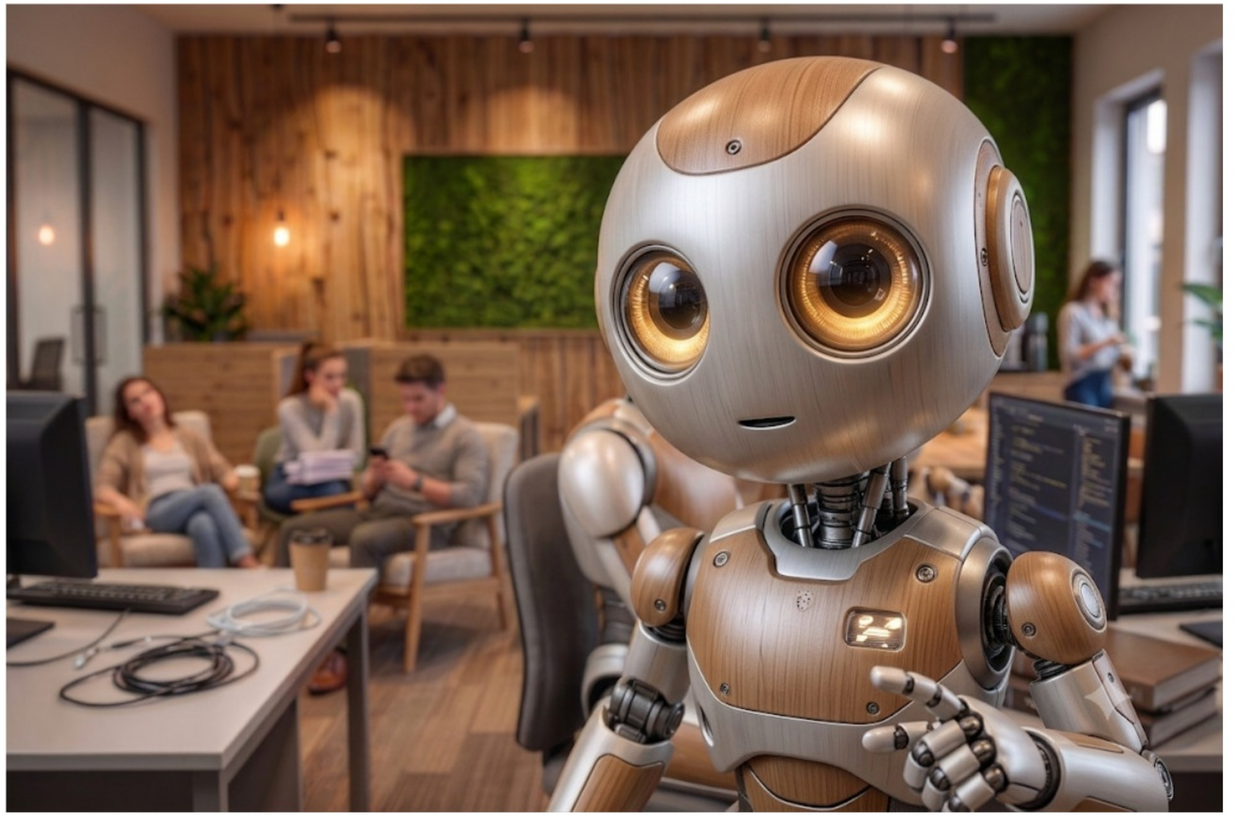




female dev club @OPEN



Jetzt ist David an der Reihe!



Danke für eure Aufmerksamkeit!

